

Project Proposal & Documentation Sheet

1. Project Title & Business Domain

Project Title	Java-Based Business Solution Proposal & Prototype
Business Domain	Subscription Delivery & Container Asset Tracking System

The system supports businesses that deliver products in reusable, trackable containers (e.g., purified water bottles, gas cylinders, or produce crates) on a recurring subscription basis. It merges two operational needs into one platform: managing customer subscription cycles and tracking the physical circulation of container assets between the warehouse, delivery personnel, and customers.

2. Target User & Problem Statement

Target User

Small to medium-sized subscription delivery businesses and their staff, specifically

- **Admin** — owns the system; manages customers, plans, subscriptions, deliveries, containers, drivers, payments, reports, and user accounts.
- **Delivery Personnel (Driver)** — field worker; views daily assigned routes, processes deliveries and container exchanges, and reports damaged/missing containers.
- **Customer** — subscriber; views active subscriptions, requests new subscriptions or plan changes, tracks deliveries, checks billing/balance, and updates contact details..

Problem Statement

Small delivery businesses using physical logbooks or spreadsheets face severe operational bottlenecks:

- **Lost Capital Assets:** Reusable containers go untracked or unreturned, incurring high replacement costs that erode profit margins.
- **Financial Leakage:** Miscalculated deposit fees, forgotten billing, and unrecorded credit balances lead to immediate revenue loss.
- **Fulfillment Failure:** Lack of centralized subscription tracking causes missed delivery windows and inefficient routing.
- **Zero Accountability:** Management lacks real-time visibility into driver routes, container conditions, and customer holdings.

3. Core System Features & Logic Flow

Core Features

- Role-based login, customer self sign-up, and admin user/role management
- Customer, subscription plan, and subscription CRUD with validation

- Delivery scheduling, driver assignment, container assignment, and status tracking
- Container inventory, condition monitoring (GOOD → UNUSABLE), and status lifecycle
- Payment recording with overpayment blocking, balance computation, and voiding
- Tabular reports for deliveries, subscriptions, and containers

Logic flow (variables, operators, decision-making):

- **Variables:** IDs (custSeq, subSeq, deliverySeq), quantity, unitPrice, totalAmount = unitPrice × quantity, amountPaid,

remainingBalance = totalAmount – amountPaid, status enums, and scheduled dates.

- **Operators:** arithmetic (totals, balances), comparison (quantity 1–1000, amount ≤ balance, date ≥ today), and logical

checks (email regex, CNT-0000 code format, unique usernames/licenses).

Operators Used

- **Relational operators** (==, !=, >, >=, <) — e.g., a delivery is accepted only if scheduledDate >= today; a quantity is valid only if 1 <= quantity <= 1000; a payment is accepted only if amount <= remainingBalance; a subscription counts as "mine" only if subscription.customerId == loggedInCustomerId; a driver sees a delivery only if delivery.driverId == myDriverId.
- **Logical operators** (AND, OR, NOT) — e.g., a container can be assigned only if status == AVAILABLE OR status == RETURNED; a delivery is marked DELIVERED only if driverAssigned == true AND containerCount > 0; deletion is blocked if hasSubscriptions OR hasDeliveries; a role update is rejected if target == self OR (target == lastAdmin AND newRole != ADMIN); login succeeds only if username matches AND password matches.
- **Arithmetic operators** (+, −, *) — e.g., totalAmount = unitPrice * quantity when a subscription or plan change is computed; remainingBalance = totalAmount – amountPaid after every payment or void; amountPaid = amountPaid + payment on collection (and − payment on void); ID counters advance by seq++ for every new customer, subscription, delivery, container, driver, payment, and user record.

Decision making:

Login routes by role (ADMIN → main menu, CUSTOMER → self-service portal, DELIVERY_PERSONNEL → field portal); status transitions follow rules (e.g., only SCHEDULED/CANCELLED deliveries can be deleted, DELIVERED auto-marks its containers, a driver is released when a delivery completes); deletes are blocked when records are linked (e.g., a customer with subscriptions cannot be deleted); and every invalid choice loops back with an error message instead of crashing.

4. System Process Specifications

TITLE	Subscription Delivery & Container Asset Tracking System
USERS	• Admin — full management and reporting. • Delivery Personnel — route execution and container exchange in the field. • Customer — self-service subscriptions, tracking, and billing.
PROCESSES	1. Account sign-up/login with automatic role-based menu routing. 2. Customer registration and profile updating. 3. Subscription plan creation and subscription issuance/plan changes. 4. Delivery scheduling with driver and container assignment. 5. Delivery fulfillment and empty-container collection (DELIVERED/RETURNED). 6. Container condition inspection and damage/missing reporting. 7. Payment collection, balance tracking, and billing history. 8. User account creation and role updating by admin. 9. Delivery, subscription, and container reporting.
SCENARIO	A customer signs up and receives a customer record; the admin issues a "Premium 5-Gal × 4 units" subscription; a delivery is scheduled and a driver plus four containers are assigned; the driver views the route in the Driver Field Portal, marks the delivery DELIVERED (containers flip to DELIVERED), and later collects empties (RETURNED); one container is found cracked and reported as MAJOR_DAMAGE (auto-flagged DAMAGED); the customer pays partial amounts and watches the remaining balance drop to zero in the billing history.